

Report 09

Frontend Development: React PWA, Marine Map & Evidence Panel

Project:	JalJeev — Agentic Marine Intelligence System
Phase:	Phase 0, Week 2–4
Focus:	Web client (React + Vite + TypeScript + MapLibre), UI components, production build

1. Objective

Build a web frontend that serves two audiences: (1) judges evaluating the system's capabilities at SIH, and (2) any browser user (DM officer, researcher, fisherman with a smartphone browser). The PWA is the primary demo surface; the native Android app handles the at-sea use case.

2. Technology Stack

Layer	Choice	Justification
Framework	React 18.3	Industry standard; excellent component ecosystem
Language	TypeScript 5.5	Type safety across geo/data handling
Build tool	Vite 5.4	10× faster HMR than CRA; native ESM
Map engine	MapLibre GL JS 4.x	Open-source; WebGL; deck.gl compatible
State	Zustand 4.5	Lightweight, TypeScript-first, no boilerplate
Data fetching	TanStack Query 5.x	API caching + background refetch
Charts	Recharts 2.12	React-native composable charts
Styling	Tailwind CSS 3.4	Utility-first; consistent design system
i18n	i18next + react-i18next 23.x	Lazy-loaded translations for 10+ languages

3. Component Architecture

3.1 Core UI Components

Component	Purpose	Data Source
ChatPanel	Natural-language query interface (voice + text)	POST /chat
MarineMap	MapLibre GL map with H3 overlay, PFZ zones, marine status	POST /marine/route
EvidencePanel	Factor breakdown, source citations, data freshness	EvidenceReceipt from response
VesselSelector	Choose vessel class	Local state
PassagePanel	Port-to-port passage planner interface	POST /marine/passage
SafetyMonitor	Live safety watch — periodic quick-check	GET /marine/quick-check
App.tsx	Shell layout, tab routing, global state	—

3.2 ChatPanel — The Primary Interface

The chat panel is the fisherman's entry point:

1. User types or speaks a question in any language
2. Optional voice input via Web Speech API (browser-native English) or Sarvam AI STT (Indian languages)
3. Query sent to POST /chat with {message, language, vessel_class, lat, lon}
4. Response rendered with risk level badge, evidence receipt (expandable), map update, voice output (optional)

3.3 MarineMap — Geospatial Visualisation

Built on MapLibre GL JS with multiple overlay layers:

Layer	Visual	Data
H3 Risk Grid	Colour-coded hexagons (green → amber → red)	Per-cell risk scores
Route Path	Coloured polyline with per-leg risk gradient	Route Agent output
Passage Corridor	H3 cells forming the A* corridor	Passage Agent output
PFZ Zones	Orange/amber overlay polygons	MOSDAC PFZ advisory
EEZ Boundary	Dashed line at 200nm	PostGIS EEZ data
MPA Boundaries	Hatched red zones	PostGIS MPA data
Nearest Port	Pin marker	Geo Agent result

3.4 EvidencePanel — Explainability

Every recommendation includes a fully auditable evidence panel:

■ Risk Assessment: MODERATE (Score: 37/100)

Factor Breakdown:

- Wave height (1.8m vs 2.0m limit): +12
- Wind speed (9.2 m/s vs 12 m/s limit): +8
- Cross-swell detected: +10
- Current speed (0.6 m/s): +7

Sources Used:

- Open-Meteo Marine (1h ago)
- NOAA OISST (6h ago)
- Copernicus Marine (3h ago)
- IMD: unavailable (pending API key)

Data Gaps:

- Cyclone track (IMD not yet available)
- Lightning nowcast (IMD not yet available)

4. API Layer

Typed TypeScript API client (frontend/src/api.ts) with:

- chatQuery(msg, lang, vesselClass) → /chat
- getMarineState(lat, lon, vesselClass) → /marine/state
- getRoute(lat, lon, vesselClass) → /marine/route
- getPassage(from, to, vesselClass, speed) → /marine/passage
- getQuickCheck(lat, lon, vesselClass) → /marine/quick-check

All calls use TanStack Query for automatic caching, retry, and stale-while-revalidate semantics.

5. Multilingual Support

5.1 Indian Languages Registry

24 Indian languages defined with displayName (English), nativeScript, bcp47Code, and region.

Coastal-priority ordering: Odia, Bengali, Marathi, Gujarati, Malayalam, Tamil, Telugu, Kannada, Hindi, Punjabi, Assamese, Urdu — then remaining Schedule VIII languages.

5.2 Voice Input via Sarvam AI

The frontend integrates with the backend's /speech/transcribe endpoint, which proxies to Sarvam AI's STT service with:

- **Maritime keyword boosting:** "Paradip", "PFZ", "trawler", "cyclone", "EEZ" as keyterms
- **Post-transcription corrections:** Phonetic fixes for common mishearings ("para deep" → "Paradip")
- **Support for 24 Indian languages** including Odia, Bengali, Tamil, Telugu, Malayalam

6. Design Canvas Prototyping

UI designs were prototyped in Claude Design Canvas before implementation:

- design-canvas/Main.dc.html — Main application layout
- design-canvas/States.dc.html — State variations (loading, error, empty, data)
- design-canvas/jaljeev-chart-interface.html — Chart and data visualisation mockups

7. Production Build

A production build exists at frontend/dist/:

- Vite-optimised bundle with tree-shaking
- Asset hashing for cache-busting
- Ready for static hosting or serving via the FastAPI backend

8. Design Decisions

8.1 Why Two Clients, Not Just PWA

The React PWA is correct for judges, desk users, and instant access with no install. But the single most operationally important feature — **continuous safety monitoring of the vessel's actual position while underway** — cannot be done reliably in a browser tab.

A backgrounded browser is throttled or suspended by the OS. A warning that arrives when the user next unlocks their phone is not a warning. That capability requires an Android foreground service — hence the native client (see Report 10).

8.2 Why MapLibre, Not Mapbox

Mapbox's billing model introduces a commercial dependency. MapLibre GL JS is the open-source fork — functionally equivalent, with no API key requirement for the base library, no billing risk, and full WebGL deck.gl compatibility.